

Sorting & Collections

Mastering the Java Collections Utility Class and ArrayList Manipulation

CodeHive Academy

1. Introduction to Sorting

Sorting is one of the most fundamental operations in software development. In Java, the `java.util.Collections` class provides a static method called `sort()` that allows you to organize elements in a list efficiently.

The Alphabetical Example

```
import java.util.ArrayList;
import java.util.Collections;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> cars = new ArrayList<>();
        cars.add("Volvo");
        cars.add("BMW");
        cars.add("Ford");
        cars.add("Mazda");

        Collections.sort(cars); // Sort elements

        for(String car : cars) {
            System.out.println(car);
        }
    }
}
```

In the code above, the list is modified in-place. Because "BMW" starts with 'B', it moves to index 0, followed by "Ford", "Mazda", and finally "Volvo".

2. Sorting Numeric Data

The `Collections.sort()` method works for all types that implement the `Comparable` interface, including `Integer`, `Double`, and `Long`.

```
ArrayList<Integer> numbers = new ArrayList<>();
numbers.add(100);
numbers.add(10);
numbers.add(50);

Collections.sort(numbers); // Sorts 10, 50, 100
```

Note: When you sort a list of Strings that contain numbers (e.g., "10", "2", "1"), they will sort lexicographically: "1", "10", "2". Always use numerical types if you want numerical logic!

3. Change the Order

Sometimes you need your data in descending order. You can achieve this by passing a **Comparator** to the sort method.

```
// Sorting in Reverse Order  
Collections.sort(cars, Collections.reverseOrder());
```

Other Useful Collection Methods

- `Collections.reverse(list)`: Simply flips the current order without sorting.
- `Collections.shuffle(list)`: Randomizes the order (great for games).
- `Collections.replaceAll(list, oldVal, newVal)`: Mass update.

4. Custom Object Sorting

If you have a list of Objects (e.g., a `Student` class), Java doesn't know whether to sort by Name, ID, or Grade. To fix this, your class must implement `Comparable`.

```
public class Car implements Comparable<Car> {  
    String model;  
    int year;  
  
    public int compareTo(Car other) {  
        return this.year - other.year; // Sort by Year  
    }  
}
```

The `compareTo` method returns:

- **Negative:** This item comes first.
- **Zero:** They are equal.
- **Positive:** The other item comes first.

5. Sorting with Lambdas

Since Java 8, you don't always need `Collections.sort()`. You can call `.sort()` directly on the list and use a Lambda expression for ultimate flexibility.

```
// Sorting by length of the string
cars.sort((a, b) → a.length() - b.length());

// Using Method References
cars.sort(String::compareToIgnoreCase);
```

This approach is cleaner and allows you to define different sorting rules on the fly without modifying the original Class.

6. Under the Hood: Algorithmic Efficiency

What happens inside `Collections.sort()`? Java uses **Timsort** (for objects) and **Dual-Pivot Quicksort** (for primitives).

- **Time Complexity:** $O(n \log n)$. This is the gold standard for comparison-based sorting.
- **Stable:** Timsort is "stable," meaning if two elements are equal, their original relative order is preserved.
- **Space Complexity:** $O(n)$. It requires some extra memory to perform the merge steps.

7. Troubleshooting & Best Practices

1. The NullPointerException: `Collections.sort()` will throw an error if the list contains `null` elements. Always sanitize your data first.

2. Immutable Lists: If you use `List.of("A", "B")`, that list is immutable. Trying to sort it will throw an `UnsupportedOperationException`. You must wrap it in a `new ArrayList<>()`.

8. Case Study: E-Commerce Product Filter

Imagine a list of Products. Users want to sort by "Price: Low to High."

```
class Product {  
    String name;  
    double price;  
}  
  
// Execution:  
inventory.sort(Comparator.comparingDouble(p → p.price));
```

By using the `Comparator.comparing` utility, you make the code readable and maintainable.

9. Key Takeaways

- `Collections.sort()` is the default way to sort lists in-place.
- Strings are sorted alphabetically; Numbers are sorted numerically.
- Use `Collections.reverseOrder()` for descending lists.
- Use **Comparators** or **Lambdas** for custom sorting logic.
- Ensure the list is mutable and contains no nulls before sorting.

Produced by CodeHive Academy © 2026
Java Fundamentals Series - Module #05